M

Functioneel Programmeren

Authors

Matt ter Steege

Matttersteege@gmail.com

Contents

1 Introductie tot Haskell 2

1.1 Lijsten 3

1.2 Tuples 3

1.3 Guards 3

1.4 Lokale definities 3

1.5 pattern matching 4

1.6 Types 4

2 Lijsten 5

2.1 Recursie 5

2.2 List comprehensions 5

3 Data types 5

October 30, 2026

Probeer de online Haskell compiler online: https://play.haskell.org

1 | Introductie tot Haskell

Functioneel programmeren (FP) gebruikt vaak recursie in plaats van loops. In plaats van for- en while-loops laat je een functie zichzelf herhalen totdat een basisgeval is bereikt. Pattern matching betekent dat je data direct op zijn vorm controleert, bijvoorbeeld leeg of niet-leeg. Je vervangt daardoor veel if/else-constructies door duidelijke gevallen per datastructuur. In FP werk je vooral met expressies die een waarde teruggeven, in plaats van statements die alleen acties uitvoeren. Functies zijn "first-class citizens", wat betekent dat je ze kunt doorgeven, opslaan en teruggeven zoals gewone waarden. Daardoor kun je functies combineren tot nieuwe functies en wordt code meer modulair opgebouwd.

Een kort voorbeeld van Haskell:

Code:

```
1 sumUpTo :: Int → Int
2 sumUpTo 0 = 0
3 sumUpTo n = n + sumUpTo (n-1)
4
5 main = print (sumUpTo 3)
```

Output:

```
6
```

Maar wat gebeurt hier nou? Bekijk het zo: in regel 4 `main = print (sumUpTo 3)` wordt een function aanroep gedaan en main wordt hier gelijk aan gesteld. het C# equivalent zou zo zijn: `Console.WriteLine(sumUpTo(3))`. Nou zie je 2 verschillende functies van `sumUpTo`, de interpreter kijkt van boven naar beneden en zoek de eerste passende functie. de parameter "3" voldoent niet aan de case "0", maar wel aan de case "n". Dus de uitkomst van `sumUpTo 3 = 3 + sumUpTo 2`. Dit blijft zich herhalen totdat de basecase "0" wordt bereikt, want deze geeft heeft geen functie in zijn definitie. Het volledige genomen pad gaat dus als volgt:

$$\begin{aligned}\text{sumUpTo } 3 &\to 3 + \text{sumUpTo } 2 \\ &\to 3 + 2 + \text{sumUpTo } 1 \\ &\to 3 + 2 + 1 + \text{sumUpTo } 0 \\ &\to 3 + 2 + 1 + 0\end{aligned}$$

Type signature

Dan moet ik alleen de eerste regel nog uitleggen: `sumUpTo :: Int → Int`, deze regel beschrijft de "Type signature". Dit geeft aan hoe de functie aangeroepen moet worden. De laatste type geeft aan wat de return type wordt (`Int`) en wat de parameters zijn (`Int`). Stel nou dat we dit hebben: `sumUpTo :: Int → Bool → String → Int`, dan hebben we dus 3 parameters, een `Int`, `Bool` & `String` en de return type is nog steeds `Int`. Maar dit staat nog niet helemaal vast, dat ga ik in de volgende subsectie nog even extra uitleggen.

Type variabeles

Types zijn op verschillende manieren te definiëren, je hebt de standaard `Int`, `Bool`, `String`, ..., maar je hebt ook "type variabeles" dit zijn types die niet specifiek gedefineerd zijn. Neem het voorbeeld `replicate Int → a → [a]`. De eerste parameter is van het type `Int`, de tweede parameter is van type `a`, wat het type van de aanroep is maakt niets uit, dit kan elke type zijn wat je wilt. De return value is een list van type a. Dus als je de aanroep doet: `replicate 10 "replicate"`, dan krijg je een string list terug. De interpreter zegt dan eigenlijk: "in deze aanroep lezen we type `a` als `String`"

Lijsten

list worden op de volgende manier genoteerd: `[1, 2, 3]` of `[1 .. 5]` (een list met de waarde 1 t/m 5). Basis functies zijn `length [1 .. 5]` is 5. Functies zoals `sum [Int] → Int`, `reverse [a] → [a]`, en `replicate Int → a → [a]` zijn functies die wel redelijk logisch zijn.

Wat ik nu ga zeggen is super wild, maar [] is een lege lijst, hierin zitten geen waarden en heeft lengte 0. Als je hier nou later een waarde aan wilt toevoegen, dan doe je dat door middel van `1 : [] = [1]` of `2 : [1] = [2,1]` hierop kan je verschillende basisfuncties uitvoeren:

```
null [1,2,3] -- Is de list leeg?
> False
head [1,2,3] -- Geef de eerste waarde van de array terug
> 1
tail [1,2,3] -- Geef alle de list zonder de eerste waarde terug.
> [2,3]
```

Lijsten hebben een onbekende lengte, maar wel een uniform type

Tuples

Tuples daarentegen hebben een bekende lengte, maar bestaan (waarschijnlijk) uit verschillende types. Je moet bij het gebruik van tuples bedenken dat deze nogsteeds één object zijn. Je behandeld ze dus hetzelfde als een `Int` oid.

Maken van Tuples:

```
remainder :: Double → (Int,Double)
remainder x = let i = floor x
              in (i, x - fromIntegral i)
{- remainder 6.56 -→ (6, 0.56)
   Je krijgt dus een object terug met 2
   waardes een Int en een Double -}
```

Extraheren van Tuples:

```
dist (px, py) (qx,qy) = sqrt (xDiff + yDiff)
    where
        tpl = squareBoth (px - qx, py - qy)
        squareBoth (xD,yD) = (xD*xD, yD*yD)
        (xDiff, yDiff) = tpl
```

De rechter ga ik een beetje toelichten. Stel dat je `squareBoth (3,4) = (9,16)`. Dan wordt `tpl = (9,16)` *--nog steeds 1 object*. Dan onderaan wordt `(xDiff, yDiff) = (9,16)`. Nu kan je xDiff en yDiff vrij gebruiken als losse functies.

Guards

Hoewel Haskell if statements heeft in het format `if condition then expression else expression` moet je dit gelijk vergeten, dit schaalt heel slecht, daarom gebruiken we guards ("|"). Deze bereiken hetzelfde als If statements, maar lezen veel beter, ze zijn te vergelijken met de switch statement in C#. Guards werken natuurlijker, je gaat namelijk weer van boven naar beneden, de eerste check die voldoet geeft aan wat de return waarde is. Bij n = -0.3 is de return waarde -1. De tweede is veel makkelijker te lezen!

If:

```
sign n = if n < 0
             then -1
             else if n == 0
                 then 0
                 else 1
```

Guards:

```
sign n | n < 0     = -1
       | n == 0    =  0
       | otherwise =  1

-- kijk eens naar deze lege ruimte :O
```

Lokale definities

Ook dit zijn weer functies. Deze kan je in een andere functie definiëren en hebben toegang tot de variabelen in die parent-functie. Hieronder zijn 2 voorbeelden die in werking hetzelfde zijn. Deze functies zijn er om code-repetitie te verminderen en leesbaarheid te vergroten.

Where:

```
distance px py qx qy = sqrt (xDiff + yDiff)
    where
        xDiff = square (px - qx)
        yDiff = square (py - qy)
        square z = z * z

-- Dit gebruik je als je van boven naar beneden
      denkt
```

Let in:

```
distance px py qx qy =
    let xDiff = square (px - qx)
        yDiff = square (py - qy)
        square z = z * z
in sqrt (xDiff + yDiff)

-- Dit gebruik je als je van beneden naar boven
      denkt.
```

Je zegt namelijk de dat de functie xDiff (zonder parameters) gelijk is aan de functie square met de waarde $(p_x - q_x)$. En elke keer als je xDiff "aanroept" krijg je dus die waarde. Het makkelijke hiervan is dat je geen parameters mee hoeft te geven, want de waardes die je wilt gebruiken (px, qx, ...) zijn al beschikbaar in de scope van de lokale functie. Let wel op dat de functies **wél** gevoelig zijn voor uitlijning (zelfde kolom, dus 2 tab's).

pattern matching

Pattern matching is het systeem dat je gebruik bij functie-aanroepen. Het kijk van boven naar beneden en zoekt naar de juiste functie voor de aanroep.

Lang geschreven:

```
conj :: Bool → Bool → Bool
conj True True = True
conj True False = False
conj False True = False
conj False False = False
```

Kort geschreven:

```
conj :: Bool → Bool → Bool
conj True True = True
conj _ _ = False
-- Gebruik "_" als je niets geeft om de waarde.
-- Je zegt dus eigenlijk, True True → True,
      anders False.
```

Types

Een type is een collectie van gerelateerde waardes. En elke expressie heeft een type, deze wordt geschreven als :: type, bijvoorbeeld (`True :: Bool`, `'a' :: char`, `[1, 2] :: [Int]`. Dit geldt ook voor functies zoals `1 + 2 :: Int`.

Basic types zijn `Bool`, `Char`, `Int`, `Integer`, `Float`, `Double`. Dan heb je nog compound types, deze zijn samengesteld. Denk hierbij aan lists en Tuples (met een maximale arity van 62).

Inferring the type of ...

Als je een type wilt afleiden, dan volg je het volgende stappenplan.
1. Zorg dat er geen overlap zit tussen de type variabelen
2. Als `f :: A → B`, in `f e` moet `e :: A`
3. Het resultaat van `f e :: B`

Voorbeeld (`map id ::`):

Gegeven: `map :: (a → b) → [a] → [b]` en `id :: c → c`. `map` verwacht als eerste argument een functie van type `a → b`. Omdat we `id` doorgeven, moeten deze types gelijk zijn: $a \to b = c \to c$. Dus volgt:
$a = c$ $b = c$. Vervang deze in het type: `map :: (c → c) → c → c` Het eerste argument (`id`) is nu ingevuld, dus blijft over: `map id :: [c] → [c]`

Type classes

type classes zijn een manier om aan te geven dat een functie alleen op een bepaalde groep types werkt. Dus bijvoorbeeld de type class "Num" is een overkoepelende groep voor Int, Interger, Double en Float. Dit is handig voor functies die alleen werken voor een bepaalde groep. De functie `(+) :: Num a ⇒ a → a → a` (de + operator) werkt alleen op types die Num zijn. Dat is ook de `Num a ⇒`, deze geeft aan dat de type variable a op zijn minst een numerieke waarde moet zijn.

Basis type classes zijn: `Num` voor numerieke types, `Eq` voor types die gelijkheids checks ondersteunen, `Ord` voor types die is een bepaalde volgorde hebben. Int's hebben een volgorde, namelijk $0, 1, 2, 3, 4, \ldots$, als je een

volgorde hebt, dan kan automatisch dingen gebruiken zoals > en <=, omdat je kan checken wat de posities in volgorde zijn tussen de waarde voor en naar de <. `Show` om dingen om te zetten in strings. En er zijn er nog veel meer.

2 | Lijsten

Recursie

Om recursie te begrijpen, moet je eerst recursie begrijpen. Een Haskell voorbeeld is:

```
fac 0 = 1
fac n = n * fac (n - 1) -- Roept zichzelf aan.
```

Hutton's recept om een recursieve functie te schrijven bestaat uit vijf stappen:
1. Definieer het type van de functie.
2. Bepaal alle mogelijke gevallen (patterns).
3. Definieer de basisgevallen.
4. Definieer de recursieve gevallen door het probleem te verkleinen.
5. Vereenvoudig en generaliseer de definitie.

laten we een voorbeeld doorlopen met de sum functie:

Stap:
- Definieer het type: `sum :: [Int] → Int`
- Alle cases: `sum [] = _` en `sum (x:xs) = _`
- De base case: `sum [] = 0`
- De andere case(s) `sum (x:xs) = x + sum xs`
- Vereenvoudig `sum :: Num a ⇒ [a] → a`

Wat doet het

```
<- List van Int's erin, Int eruit
<- lege list, of niet-lege list
<- termineert met een lege list
<- anders voorste element + sum van de rest
<- uiteindelijke type Num a (Int, Float, Double,...)
```

List comprehensions

In Haskell is er de `[expr | x ← list]` notatie. Deze notatie bouwt een nieuwe lijst van een oude lijst.

één extra:

```
addone :: Num a ⇒ [a] → [a]
addone xs = [x + 1 | x ← xs]
```

Deze functie zegt eigenlijk hetzelfde als "Voor elke x in xs, return x + 1". Elke waarde in de lijst word dus één groter. **Som van even:**

```
-- Check is a number is divisible by 2
even :: Integer → Bool

sumeven :: [Integer] → Integer
sumeven xs = sum [x | x ← xs, even x]
```

Deze functie `[x | x ← xs, even x]` zegt het volgende: maak een array van elke x in xs, waar `even x == true`. Dus deze list comprehension zet een lijst zoals `[1, 3, 4, 6, 7]` eerst om in een de lijst `[4,6]` en daarna roept hij `sum` erop, wat uitkomt op 10.

3 | Data types

Stel nou dat je zelf een datatype wilt maken, dan kan je dat zo doen:

```
data Direction = North
               | South
               | East
               | West
```

Het keyword `data` geeft aan dat er een nieuw type gemaakt wordt. De naam van het type "Direction" moet beginnen met een hoofdletter. En dan één of meerdere constructors achter de "=" gesplits door "|". Stel nou dat je `> :t North` doet om het type van North te krijgen, dan krijg je dat North van het type Direction is (idem met lists van North/South/...).
Hier kan je vervolgens ook functies voor schrijven:

```haskell
directionName :: Direction → String
directionName North = "N"
directionName South = "S"
directionName East = "E"
directionName West = "W"

--directionName North → "N"
```

Zo zijn dit voorbeelden van ingebouwde data types:

Bool:
```haskell
data Bool = False | True
```

Int:
```haskell
data Int = ... -1 | 0 | 1 ...
```

Char:
```haskell
data Char = ... 'A' | 'B' ...
```

Echter worden de "oneindige" types (Int, Char,...) wel apart behandeld door de compiler, maar dit is het idee er achter.

maar deze data types slaan inhoudelijk geen data op, deze geven alleen aan wat ze zijn. Om data op te slaan kan je een data type zoals "Point" maken: `data Point = Pt Float Float`. Dit moet je lezen als: Maak een nieuw dataype `Point`. Om een Point te maken gebruik je de constructor `Pt` en deze heeft 2 Floats nodig. Een voorbeeld zou zijn `Pt 2.0 5.0`. (Je had ook `data Point = Point Float Float` mogen schrijven, maar Pt is korter). Wat nou als je de afstand van de oorsprong $(0, 0)$ wilt berekenen, dan kan je dat met een functie zoals dit doen:

```haskell
norm :: Point → Float
norm (Pt x y) = sqrt (x*x + y*y)
```

Je gebruikt hier `(Pt x y)` omdat dat de vorm is die je hebt gedefinieerd hiervoor. Je moet het wel tussen haakjes zetten, omdat dit één type/argument is en niet meerdere.

Zo'n constructor is eigenlijk gewoon óók een functie (wat een mindfuck), want je zou het namelijk ook zo kunnen lezen: `Pt :: Float → Float → Point`. En als je dan met precies dezelfde syntax deze functie aanroept: `Pt 2 5` krijg je dus weer een punt met coordinaten 2 & 5.

Maar het kan nog interessanter. Want één datatype kan meerdere constructors hebben, zoals:

```haskell
data Shape = Rectangle Point Float Float -- linksboven, breedte, hoogte
           | Circle Point Float          -- Midden, Radius
           | Triangle Point Point Point  -- Hoek A, Hoek B, Hoek C
```

Hier kan een "Shape" dus 3 verschillende vormen voorstellen. Dit heet een Sum type (of variant type). Deze variant types kan je vervolgens dmv pattern matching ook weel uit elkaar halen bij een functie. Elke vorm heeft namelijk een oppervlakte (ga ik even gemakshalve vanuit). Dus dan kan je een fuctie schrijven die een Shape als argument geeft en een Float terug geeft.

```haskell
area :: Shape → Float
area (Rectangle _ w h) = w * h -- Wanneer shape een rectangle is (hoekpunt boeit niet)
area (Circle _ r) = pi * r ^ 2 -- Cirkel, midden boeit niet
area (Triangle x y z) = ...    -- Driehoek, iets te lange formule
```

Datatypes kunnen zichzelf ook hebben in de constructor: `data IntTree = EmptyTree | Node Int IntTree IntTree`. Dan is het wel nodig om een EmptyTree te declareren, want anders kan je nooit een eindige boom maken.